

Who Can Turn Off Your Ethereum App?

A Full Stack Walk Away Test

James Carnley

Ethereum File System



Building Cypherpunk Apps

Why make an app you can walk away from?

What happens when an app disappears?

Users lose their work and trust

Other developers can't rely on that functionality

Communities splinter

Software worth building on

For people

Keep years of useful work

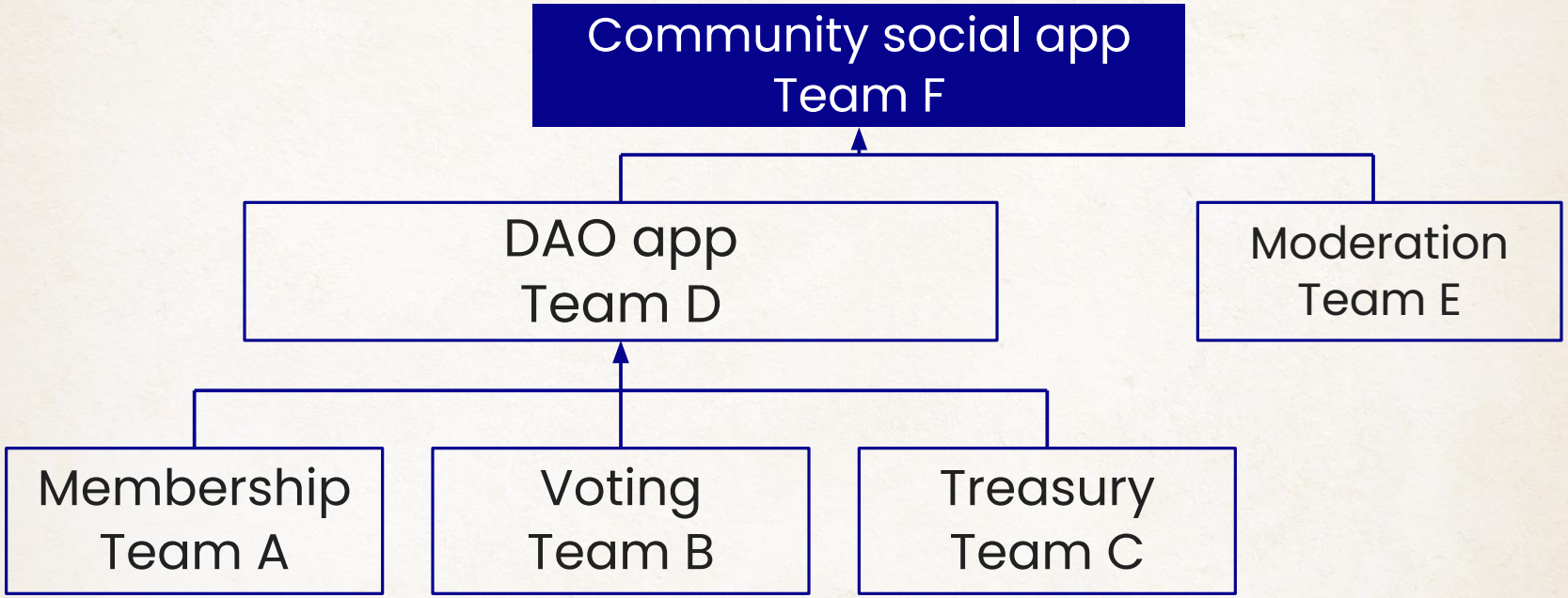
Choose better tools

For builders

Reuse dependable capabilities

Build bigger things together

Hyperstructures: build on each other



Independent teams work together via stable interfaces

A walk away test

Single points of failure and control

The team disappears

Can users keep working?

Can others run the services?

Control becomes hostile

Can users keep their work?

Can they keep the rules they chose?

TEST

Choose one important user task.
Try it without your team’s services or help.

The application stack

Interface

- Names and discovery
- Runnable software
- Accounts and recovery

Data

- Storage and formats
- State and indexes
- Background work

Rules

- Contracts and governance
- Network and settlement



Interface

Finding and using the app

Find a saved release

Run it without your servers

Recover access without your help

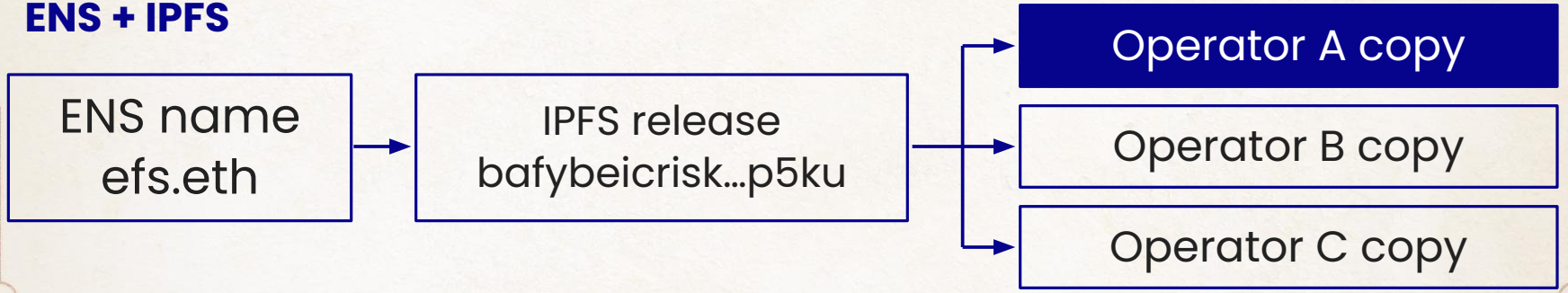
Can people find and use your app?

More than one way into the app

DNS + HTTP



ENS + IPFS



IPFS gateways: efs.eth.limo (or .link) or your own IPFS node

TEST Block the usual website. Can you open a trusted release through another route?

A complete runnable release

Keep every file the client needs to start

Incomplete release

index.html on IPFS

↓

Team server: app.js
Offline

Client cannot start

Complete release

IPFS bundle
index.html
app.js + styles + assets

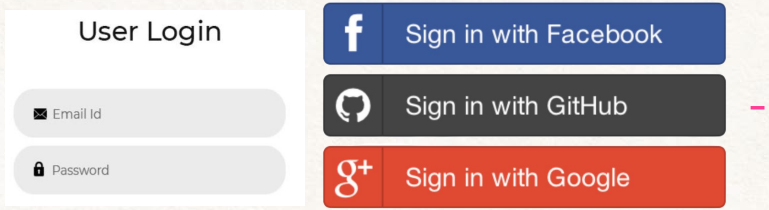
↓

Client starts

TEST Block your team's server, then reload the client.
Does it start?

Access without the company

The client starts. Can the user actually get in?

App → 

--> Locked out

An account another service can recognize

User's wallet → SIGN IN WITH ETHEREUM → Same account and permissions

TEST Disable company login. Can the user complete the same action through another client?



Data

Continuing the work

Is the data portable?

Keep the files and their meaning

Use them through another app

Let others run the supporting services

Data another app can understand

Example shared record

```
{
  id: note-17
  author: 0xDEADBEEF…
  text: Meeting at 10
  body: Review proposals.
  file: agenda.md
  vers: 2
}
```

Shared meaning

What each field means

Who can change it

How versions fit together

Who can read and decrypt it

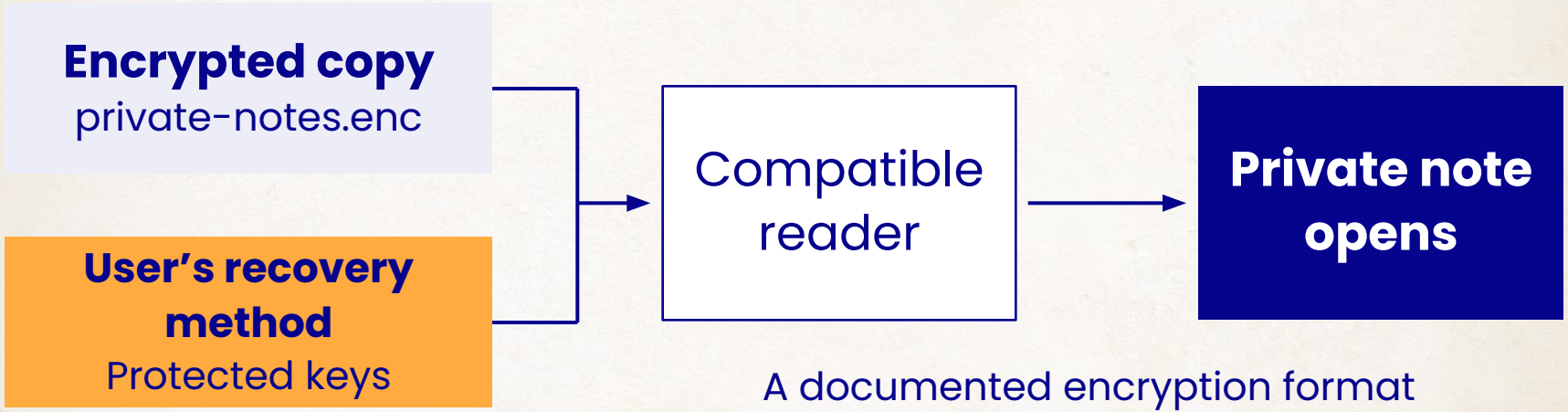
Shared schemas with EAS  Documented exports from your database.

TEST

Hand over an export and its format.
Can another developer understand it without asking you?

Private data you can recover

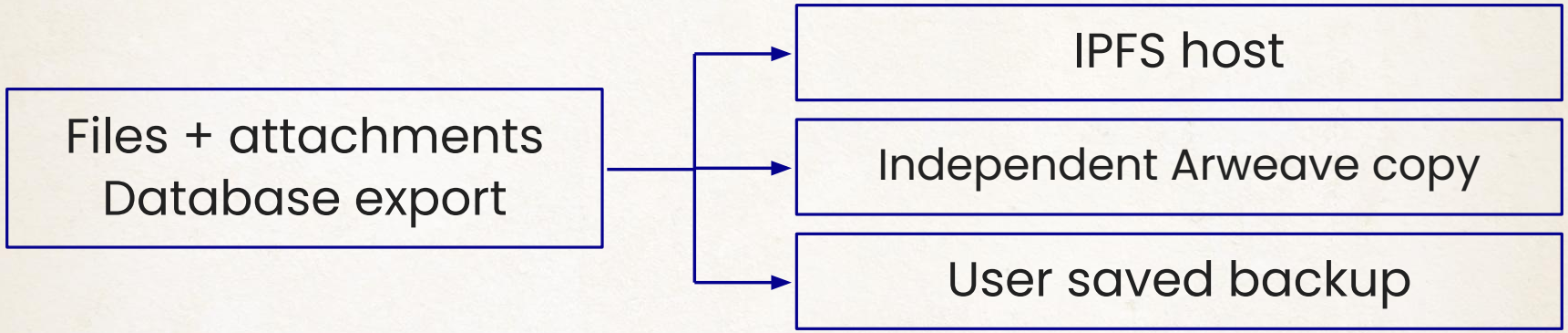
Keep private work readable without company approval.



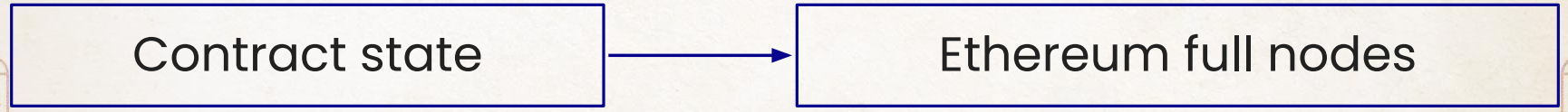
TEST With the company offline, recover the keys and reopen a private file in another reader.

Who keeps a usable copy?

Files and databases



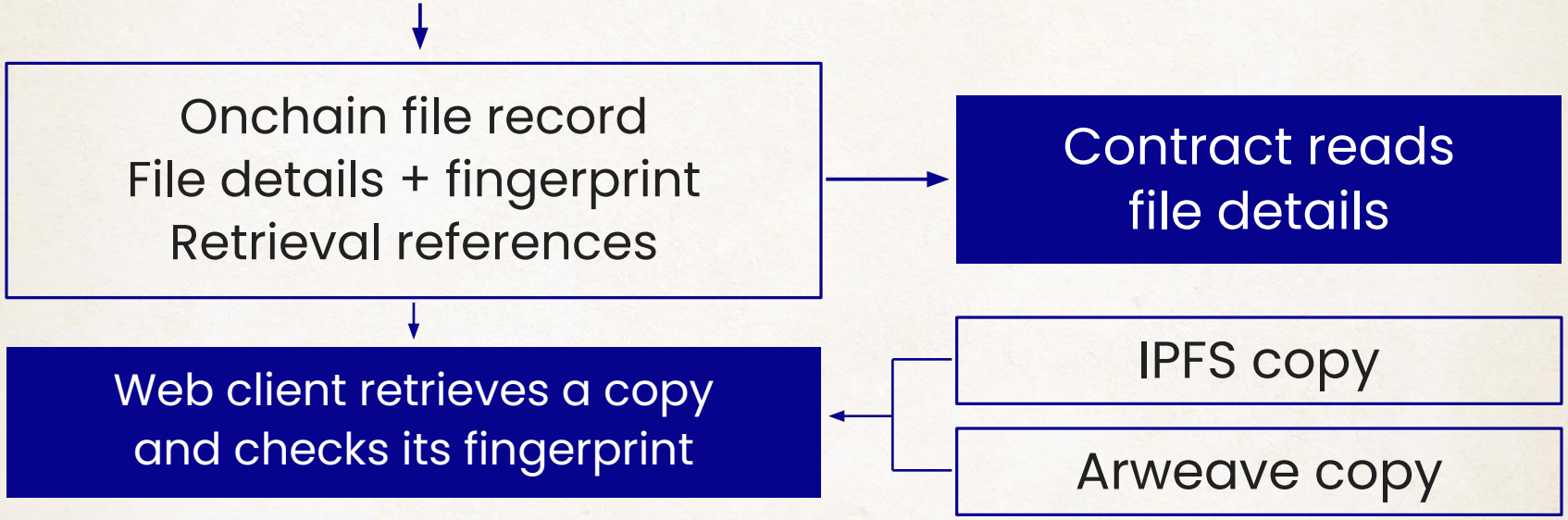
Onchain records



TEST Block the primary storage provider.
Can you recover the record and attachments elsewhere?

/EFS/ Shared paths, data verifiability

Example path: /notes/meeting.md



TEST Fetch from another source. Does its fingerprint match the accepted file record?

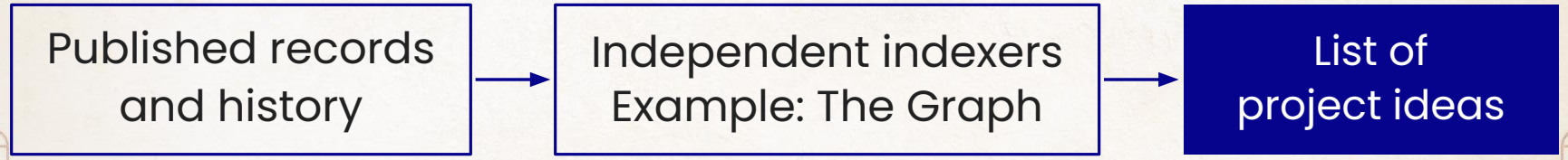
The data survives. Can your app read it?

Read a file record



Light clients such as Helios can verify supported chain reads.

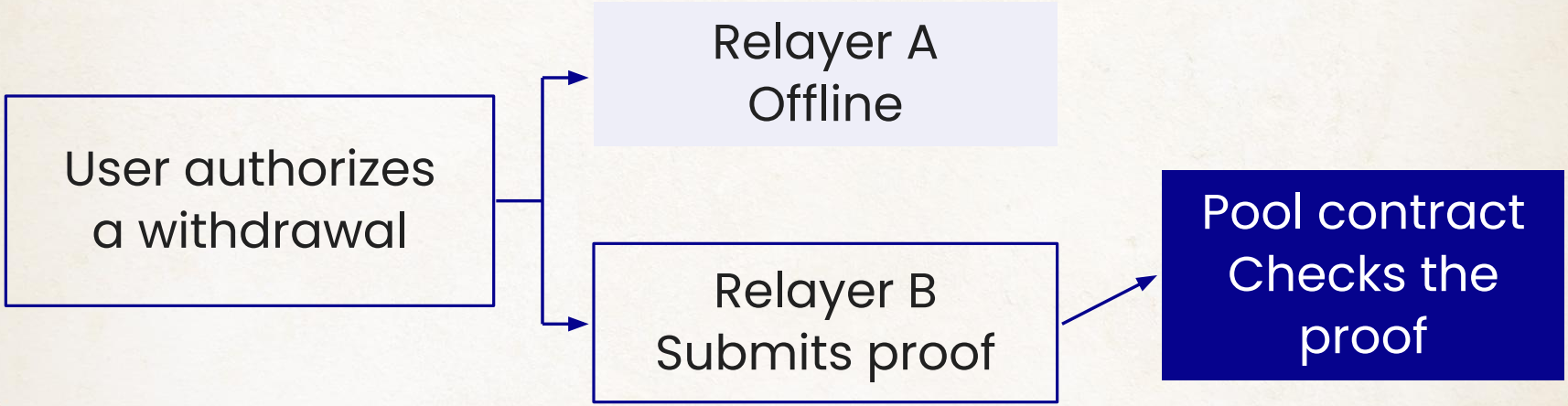
Working with large amounts of data



TEST Switch the RPC, then the index. Can users still find their notes and latest changes?

Who sends the next transaction?

Tornado Cash: replaceable relayers



TEST Stop your transaction service. Can another operator complete the user's request?



Rules

Keeping dependable rules

Stable core contracts

User controlled app upgrades

Clear rules for shared governance

Who chooses the changes?

Immutable contracts

UNIX mindset: Do one thing and do it well

A stable core

Keep contract code fixed

Let other apps build on it

Room to improve

Deploy a new version

Let users choose to migrate



Uniswap: fixed code, shared exchange logic

TEST

Keep the old app release and contract address.
Can users complete the task without upgrading?

The network beneath the app

Ethereum and L1

Use another node

Keep required history

Check inclusion and finality

L2 and settlement

Who orders transactions?

Can users submit through L1?

How can users withdraw?

TEST

Rehearse a documented network outage.
Can the user complete the recovery steps?

Could someone run it without you?

No private help from the team

Public handoff

software + license
data + instructions
configuration + costs

→ Independent operator

Their own keys and hosting

→ User's task completed

A working service users can choose

TEST Give an outsider the docs. Can they run it and complete the task without contacting you?



Putting this into practice

A walk away test for your app

Your walk away checklist

Interface

Open a saved release

Recover access and keys

Data

Restore usable records

Switch clients and providers

Run essential background jobs

Rules

Identify who can change rules

Keep using accepted rules

Try the chain’s outage or exit path

TEST Remove one team dependency. Repeat the user’s task. Improve what stops it, then try again.

Our work can outlive us

Thank you!

Let’s build reliable and long lasting apps together 🖖



James Carnley
Ethereum File System
efs.eth.link
JamesCarnley@gmail.com
@JamesCarnley